Jairs

4 September 2026

A Jai-inspired systems language in Rust. Incremental salsa front end, lossless CST, typed SSA mid-end, and **three** execution engines — a bytecode VM, Cranelift and LLVM — held byte-identical by a differential harness.

7/7 gates green · 1082 tests · ADR-0200 latest · ALL TWELVE WAVES DONE · everything merged to main · 5 language utilities landed

TESTS	CORPUS	ADRS	DIAGNOSTICS	EDITOR CHECKS
1129	281	200	131	189
workspace, all seven gates	jr files, all three engines	0001 to 0200, immutable	declared codes, E0296 next	Neovim 170 + Zed 19, verified not gated

The vertical slice, and what it costs to be honest about it

EXIT CRITERIA

- DONE hello.jr runs in the VM
- DONE compiles to a native binary
- DONE a **third** engine agrees (LLVM)
- DONE rustc-grade diagnostics
- DONE formatter round-trips the corpus
- DONE editor integration (Neovim)
- DONE DWARF: lines, types, locals
- OPEN verified Linux x86-64 CI run

The last one needs a push. Configured, never run, so it is a decision rather than a technical gap — and after twelve waves it is the **only** exit criterion still open.

HOW CORRECTNESS IS ESTABLISHED

All three engines run every corpus program and must agree, on output and on trap wording, from one shared formatter. The LLVM back end agreed with the VM on all 114 executable programs on its **first** run — because both native engines read the same MIR and ask `jr-pool` the same layout questions, which is what ADR-0018 §2 centralising layout bought.

Probe the premise by writing the thing. Eight times now. The sharpest was `#simd`: Cranelift's type **constructor** happily makes a 256-bit vector that no backend can compile, so a plan built on the constructor would have looked right until the first compile. Probing set the legal widths, and separately found that no ISA has an integer vector divide.

A performance claim needs a measurement, including when it says no. Parallel sema was written, worked, and produced byte-identical output at every thread count — then measured at 1.20x against a 2.5x ceiling, because 40% of a check runs inside the type pool's exclusive critical sections. Reverted. The number is the deliverable.

A plan's stated reason is checkable. W4.5 was scheduled after W4 because exhaustiveness "wants comptime type info". Three greps and a two-line program showed it does not: the enum member set is already in the pool during checking. The wave moved forward and the table records the amendment — a wave order resting on a dependency that does not exist is a plan contradicting itself.

Check the query a claim is about. "The arithmetic around a `#run` is not folded" was true of the **built** MIR and false of the **optimized** one — and the corpus only ever snapshots the built query, so nothing the tests display would have shown it. A claim about optimisation needs a probe of the optimized body.

A well-typed placeholder is invisible to every gate this project has. `t := Point;` — a type bound to a local — type-checked cleanly and **both engines exited 0**, storing an undefined value into a slot of a type with no runtime layout at all. Three of these now, and each was found by asking a question no test asks: what does this lower to? The corpus checks exit codes and snapshots the built MIR; a construct that is legal, silent and unread sits outside all of it.

A test naming an unimplemented thing has a one-wave shelf life. The refused-body test has now had its construct replaced **three** times, each because the wave after it implemented the gap it named — most recently the file-scope mutable variable, which its own comment had called "the shortest program that reaches it today". It reads an imported global now, which is deliberately not built. And one of this wave's own new tests passed vacuously until it was teeth-checked.

A count is not an enforcement. The test guarding `TrapKind::ALL` asserted `len() == 11` — which catches nothing, since a variant left out keeps the length right. Replaced with an exhaustive match, it **immediately** found four of fifteen kinds had never been in the list, in a list whose doc comment described a driver loop that does not exist. Those four had therefore never been checked for message distinctness — the property that keeps a real engine disagreement visible. Third instance of one shape: a hand-kept list, a comment claiming something enforces it, nothing that does.

A plan entry saying "blocked on X" is worth twenty minutes of checking that X exists. W12′s last item was recorded as blocked on `enable_value_labels` in Cranelift's ISA flags. That flag does not exist — not in the settings, not in the meta crate, nowhere. The real gate is one `collect_debug_info()` call, and wiring it produced ten real register ranges in twenty minutes.

The language today

WORKS, END TO END	ABSENT (WAVE)
Full integer tower, bool, string, pointers	pointer difference p - q, deferred
float32 and float64, plain IEEE-754, no traps; a Math module with vectors, Matrix4 and Quaternion	percent on floats (E0223); is_nan
struct, nominal, one level	
union, nominal, untagged — a cross-field read reinterprets	
variant — a tagged union: a wrong-case read traps, switch destructures	a recursive variant; eliding the check in an arm
enum and enum_flags, namespaced, bare dot-member, switch cases	an explicit backing type
Fixed arrays, views and [..]T dynamic arrays, bounds-checked; a length may name a constant; array literals T.[a, b] in every expression position	a length needing evaluation; an array literal at file scope (refused by name); a struct literal
struct #soa(N) — one array per field, and e[i].x means e.x[i]	a bare e[i]; using inside one
#simd [N]T — a vector at one of the six register widths; elementwise +% -% *% on integers, + - * / on floats, lane indexing, .count	any other width; integer /; comparisons (need a mask type); swizzles
Per-field #align N (a minimum, power of two up to 4096) and #place N (an exact offset, may overlap, may be unaligned)	a struct-level #align; any packing form; an operand needing evaluation

WORKS, END TO END	ABSENT (WAVE)
a file-scope mutable variable: counter: s64 = 5; shared by every procedure in the file, const-evaluated initialiser, --- reads as zero	a bare value coercing to Any
..T variadics, including ..Any — arguments are pointers	
#c_variadic — a C-convention variadic a #foreign declaration may take	a Jairs procedure being one
An aggregate crosses a #foreign boundary, by the platform C ABI, in all three engines	
A (T) -> U #c_call procedure *type*, so a body can be handed to C	a #c_call procedure inside a #run
atomic_load, atomic_store, atomic_add, atomic_compare_exchange on s64, sequentially consistent	wider types; other ops; weaker orderings; a fence
Threads: spawn, join, joinable, yield_now, and a spin lock — modules/Thread	a per-thread backtrace; Thread_Local; channels
cast and xx from context; operator overloading	unary, index and call overloading
Trapping arithmetic, wrapping variants, bitwise	transmute
if, else, while, for, break, continue, defer, using	
switch with exhaustiveness checking over an enum; else	patterns, ranges, guards; a jump table
Multiple returns, named args, literal defaults, #must on a discarded result, a multi-result call in a return	
import, foreign, system_library, #scope_module, #expand macros that splice, #modify predicates, #bake_arguments, @note metadata, and noted_count / noted_name / noted_declarations to ITERATE it	
$T and $$T procedures, Box($T) structs, $N comptime-value parameters, #expand macros that splice, #modify predicates, #bake_arguments — W5 complete	inference through Box($T); a length needing arithmetic
Compile-time run at file scope or in a body, across files; type_info(), Any, #insert, #code	a cross-file #run value; a Code value (declined)
A type as a compile-time value: T :: Point aliases one, usable anywhere Point is	a chain B :: A; comparing types; Type as an annotation
A type in a runtime position is refused — it has no representation to store	
Traps name their source line and the live call chain	a per-frame line; inlined frames (none exist)
Bounds checks, strippable by build setting or #no_abc	a per-index #no_abc; any other build setting
context, a hidden parameter passed by pointer	
Procedures as values: call, pass, return, struct field	a cross-file or foreign proc value; comparing one
null, a context-typed pointer literal; malloc and free	cast to a pointer
An allocator in the context, installed and called through	
Pointer offset p + n, n + p, p - n — element-scaled, unchecked	the difference p - q; p[n]; ordering
push_context, a block with its own copy of the context	
talloc / reset_temporary_storage — a per-context bump arena	hands out *u8 only; alignment; a growable region

No error-handling model yet — ADR-0008 reserves the slot and the first half exists (several return values); `#must` is owed its own ADR. **No GC and no RAII**, which is a design value rather than a gap. **A union is untagged**, so reading a field other than the one last written reinterprets bits: documented in three places because it cannot be diagnosed. **Indirect calls work** (ADR-0059) — this note used to call them the project's largest gap, and it went stale where nothing could catch it, which is the argument for a dashboard whose generator is committed. **The largest remaining gaps** are no longer wave-shaped, because every wave is closed: a **per-thread backtrace** (the shadow call stack is one global, so a trap in a spawned thread may name the wrong frames), a **register-resident local in DWARF** (measured reachable, needs `.debug_loclists`), a **file-scope mutable variable**, and a **typed constant** plus **qualified imports** — the last two found by building the library rather than by design review.

Compiler internals, 17 crates

STAGE	STATE	NOTE
Lexer, parser, CST, typed AST	works	Hand-written, error-recovering, trivia-preserving
Formatter	works	Pure function over the CST; has lost a construct in most waves that added a node kind — #simd made it 9
HIR, name resolution, modules	works	Flat import merge; cycles legal; export filtering
InternPool: types, values, layout	works	One layout computation and one integer evaluator, shared. Behind an RwLock: reads share, interning excludes
Sema: signatures, checking	works	131 codes, E0296 next free, ownership enforced by a cross-crate test; folds size_of, os() and type_of; no const-eval here, by design
MIR: typed SSA	works	Block parameters, not phis; explicit bounds check and zeroing
Mid-end	5 passes	Inline (non-leaf, bounded rounds), forwarding (cross-block), const-prop, DCE, plus the bounds-check strip. -O0 skips all of it
Const-eval	works	Runs MIR through the bytecode VM
VM: register bytecode, libffi	works	No JIT; indirect calls, malloc from its own region; a vector is memory and an elementwise loop
Cranelift back end	works	Aggregate returns via sret; indirect calls via func_addr; a vector is one register; .debug_line and .debug_info written by hand with gimli
LLVM back end	works	Via inkwell + LLVM 21, behind a default-off cargo feature; gate 7 is its own test run; DWARF via !dbg metadata, so none of the gimli work carries over
DWARF debug info	works	Line tables, base and struct type DIEs with real field offsets, a subprogram per function, and stack-resident locals — in BOTH back ends, from one span source
Language server	13 caps	Diagnostics, hover, goto, completion, rename, actions, hints, symbols, signature help, and semantic tokens — the last one landed in ADR-0159, so the set is complete
Neovim integration	works	Runtimepath dir, no plugin manager; 170 checks
Driver	stub	Should consume the workspace notion that now exists

Roadmap: twelve waves closed, plus the Simp programme and per-OS support — see PLAN §1.5 and §7

W1 Data	done	Numeric tower, wrapping ops, enum, enum_flags, union, arrays, views, cast, xx, operator overloading. Closed by ADR-0048.
W2 Flow and scope	done	for with it and it_index, labelled break, defer, using, multiple returns, named and default arguments, #scope_module. Closed by ADR-0054; ADR-0055 then closed a gap six corpus files had carried for eleven waves.
W3 Runtime core	done	context (ADR-0057), the bounds-check build setting (ADR-0058, finishing ADR-0003), indirect calls (ADR-0059), null plus a memory source (ADR-0060/0061), the allocator protocol (ADR-0062), push_context (ADR-0063), pointer arithmetic (ADR-0064), temporary storage (ADR-0065), and traps with backtraces (ADR-0066) — a trap names the frames that were live, byte-identically in both engines. Closed by ADR-0066; a source-level backtrace with inlined frames is deferred, because inlined frames have no runtime existence.
W4 Comptime	done	DONE in ten sub-waves (ADR-0069 to ADR-0080), delivered that way because a 10-14 week wave cannot be verified the way a one-ADR wave can. A #run may call an imported procedure and appear in a body (ADR-0069), turning two internal compiler errors into working programs. An array length may name a constant (ADR-0070), which REPLACED the scheduled aggressive const folding after probing showed const-prop already did it. A type is a compile-time value (ADR-0071), which closed a silent miscompile — a type bound to a local compiled to an undefined value in a slot with no layout at all. #insert lowers where it is written (ADR-0072/0073), in the enclosing scope, and a computed operand evaluates at compile time and splices before the point sema and the VM

		become mutually recursive. type_info(T) (ADR-0075) reads its numbers from the same layout_of every real layout decision uses, so reflection cannot disagree with the layout it describes, and Type_Info lives in modules/Basic in Jairs rather than inside the compiler. Any erases a pointer at a named boundary (ADR-0076); #code is a value the metaprogram waves consume.
W4.5 Pattern matching	done	switch with exhaustiveness checking, a bare dot-member as a case (settling ADR-0041 §2 step 5), and a tagged variant type (ADR-0067, ADR-0068). Reordered ahead of W4 after checking showed its stated dependency on comptime was a want rather than a need. The variant follows ADR-0045 §1's own instruction — a different declaration form, not a change to union — and union is untouched, still untagged and still one word smaller.
W5 Polymorphism	done	DONE in fifteen sub-waves (ADR-0081 to ADR-0097). $T inference, then $$T mixing inference with baking in one signature, then parameterised structs Box($T), then $N comptime-VALUE parameters — a length that is a constant rather than a type. #expand macros SPLICE their body into the caller's scope rather than calling it, #modify predicates run at instantiation and can REJECT one (E0275), and #bake_arguments produces a specialised procedure. The last piece lowers to a REAL procedure — a clone with the baked parameters dropped, their literals substituted and the kept ones remapped, which is the same machinery $N instantiation uses, so the wave ends on a REUSE rather than a new mechanism. Two plans were corrected by building: ADR-0096 intended to use the const-eval pre-pass and found it runs AFTER lowering, and the expansion fixed point (ADR-0120) needed a settling check because an instantiation family can fail to converge.
W6 Metaprogram	done	DONE, closed by ADR-0154. @note attaches metadata to a declaration (ADR-0098) as its own node kind rather than a generic attribute, because a note is DATA for a metaprogram while the directives are INSTRUCTIONS to the compiler. has_note and note_value read it at compile time (ADR-0099), folded in sema with no VM and no new query — unlike type_info, which folds later because it needs a layout. The first argument is the declaration itself rather than its name as text, so a misspelling is an unresolved name instead of a silent false. noted_count / noted_name / noted_declarations then let a metaprogram ITERATE what it found, and the compiler-emitted static-data table was the wave-sized architectural decision that made it possible. The wave headline claim is met: a metaprogram finds declarations by note and generates code for each one. DONE at last, by ADR-0195 — and this row said done for eleven waves before it was. Its own item list claims '#run build() build scripts replacing makefiles' and what had shipped was two SETTINGS. The rest arrived, but NOT as a #run: comptime code may call no #foreign, so a #run cannot read a file, shell out, print, or even allocate. And the deeper finding from reading 23 real build.jai files is that Jai's build power is the STANDARD LIBRARY rather than the compiler API — cloning a dependency, stamping a git hash, formatting a disk image. So a script is an ordinary program run in the VM, and the whole feature needed no #foreign_at_comptime, retiring a locked decision that called it non-negotiable BECAUSE of build scripts. modules/Compiler exists now too, so W7 is nine of nine.
W7 Stdlib	done	DONE in twenty-three sub-waves, closed by ADR-0158, and every one was driven by a refusal or a bug rather than a checklist. String exists because ADR-0099 refused == on two strings — same storage and same contents are both plausible for a {data, count} pair — and named a byte loop as the fix, which is a library job since an == that looped would be the only implicitly-looping operator in the language. Its OWN module rather than more of Basic, and the deciding argument is not size: adding to Basic would mean nothing ever tested that TWO modules can be imported at once. Then Sort, Array, Map, Math (vectors, Matrix4, Quaternion — ADR-0115 had declared Math complete when none of the three existed), Random, Time, Bucket_Array, JSON, File, File_Utilities, Process and Socket. Twenty modules. Four polymorphism defects and two silent divergences were found by USING the language rather than by testing it, which is the argument for dogfooding as the acceptance test.
W8 Performance	done	DONE in eight sub-waves (ADR-0142 to ADR-0149). An optimisation level -O0/-O1 whose real deliverable is the check the mid-end never had: every corpus program behaves identically at both levels, so a wrong answer is attributable to lowering rather than a pass. The LLVM back end via inkwell behind a default-off feature, making the differential three-way — it agreed with the VM on all 114 executable programs on the FIRST run, because both native engines read the same MIR and ask jr-pool the same layout questions. #align and #place, the first features whose whole implementation is a layout FEATURE rather than a fix. Inliner maturity: the leaf rule is gone, termination is a bounded round count, and forwarding follows a single-predecessor chain — 024-hello's optimised MIR now flattens three call layers the old pipeline could not. A published compile-throughput number, and heap_sort chosen by a COMPARISON COUNT rather than a wall clock, because this project can measure its own throughput and deliberately cannot measure the programs it compiles. #soa, where the sugar IS the feature — without e[i].x it buys nothing over writing [N]T by hand. #simd, whose legal widths were set by probing Cranelift rather than by taste, and whose integer lanes take the wrapping operators because no vector add can trap. And parallel sema, MEASURED AND REFUSED: 1.20x against a 2.5x ceiling, reverted, with the 40%-serial measurement and two named blockers left behind.
W9 Tooling depth	done	DONE, closed by ADR-0159. Semantic tokens were the one LSP capability still missing and the other thirteen had landed early; the set is now complete. The wave also RE-SCOPED its own DWARF item with evidence rather than carrying the plan's description: the plan said 'richer DWARF (locals, struct layouts)' as one line, and probing showed it is two implementations — Cranelift wants .debug_info DIEs written by hand while LLVM writes DWARF itself from metadata — so it moved to W12 where it could be budgeted honestly. VS Code stays descoped by ADR-0036, and any LSP client works unpackaged.
W10 Graphics, in Jairs	done	DONE in four steps (ADR-0164 to ADR-0167), on a foundation the plan had wrong. It was described as ALL library work with no compiler changes; PLAN §8.5 corrected that, and the corrections cost three compiler waves first: no aggregate crossed a #foreign boundary (ADR-0160/0161, a shared C ABI classification in jr-pool so all three engines agree), objc_msgSend is C-variadic on top of that (ADR-0162, the #c_variadic marker and E0289), and a library search path was needed to link SDL2 at all (ADR-0163). Then Window and a 2D surface, an event loop ADR-0164 had said was impossible — recorded and then DISPROVED by one line, because SDL_Event is a union and a *pointer to one is takeable — immediate-mode UI widgets, and Image with BMP and textures.
W11 Concurrency	done	DONE, and the LAST of the twelve (ADR-0175 to ADR-0177). The blocker was not the one this row used to name. PLAN §8.3 said a per-thread VM stack, atomics as language operations, and a comptime rule; it did not say a thread body could not be NAMED. pthread_create takes a function pointer, and #c_call was a DECLARATION attribute with no spelling in a TYPE — jr-pool had modelled the two conventions as distinct types since ADR-0001 and the checker interned the distinction away with a comment explaining why that was safe. Found by three probes in four minutes, the third reporting 'expected (s64) -> s64, found (s64) -> s64': two identical types, because the type describer did not render the convention either. Then atomics as a MIR Rvalue rather than library calls, which the exhaustive-match rule turned into nine compile errors each of which had to be argued — forwarding would have moved a write past a synchronisation point, DCE would have deleted a compare-exchange whose EFFECT is the lock. Then modules/Thread as a binding, with a spin lock because pthread_mutex_t is 64 opaque platform-sized bytes this language cannot spell. Three threads, 3000 atomic increments, exactly 3000, in both native back ends, five runs per test.
W12 Debug info	done	DONE in six waves (ADR-0169 to ADR-0174), and §8.4 had claimed line tables already existed when dwarfdump on a built binary printed an empty section. So it started from zero. A .debug_line for Cranelift written by hand with gimli — a SourceLoc indexing a (path, line) vocabulary, a relocation writer for sequence addresses — and then for LLVM, where NONE of that is reusable because LLVM writes DWARF itself from !dbg metadata. Both verified by PARSING the section the way lldb does rather than grepping dwarfdump. Items 2 and 3 turned out COUPLED: a struct mapping was written, was correct, and dwarfdump showed no struct, because LLVM prunes a type nothing

		declares and a signature is not a declaration — what retains a type is a VARIABLE of it. One item remains and is specified rather than vague: a register-resident local, whose plan entry named an ISA flag that does not exist.
Jai graphics API	done	DONE in four ADRs (ADR-0185 to ADR-0188). The Simp programme above got the SHAPE right and the SIGNATURES wrong, because it worked from documentation; these worked from Jai's own module source, vendored verbatim by two open-source projects and diffed against each other. Eight signatures were wrong. Two not cosmetically: the coordinate origin was mirrored (Jai is bottom-left y-up, the SDL renderer was top-left y-down) and every call carried a state handle Jai does not have. Removing that handle needed file-scope mutable variables in all three engines, which this plan had owed since ADR-0178 and which exposed the let-else hole in the exhaustive-match rule. Pixels now go through GL 2.1 with two real GLSL 1.20 shaders and glDrawArrays, not SDL_RenderGeometry. Two compiler defects came out of it: a constant's value is keyed by ItemId and a computed #insert renumbers those, and a default argument silently did not apply across a module boundary.
Simp programme	done	DONE in four ADRs (ADR-0179 to ADR-0182), on top of the twelve. Qualified imports first, because the graphics restructure could not be written without them: Window and File both exported open, so a program that both drew and read a file was E0211 and unwritable. Then the target OS as a compile-time value — the compiler had NO notion of an operating system anywhere, its whole notion of a target being two numbers in TargetLayout — and one library use of it, Time's CLOCK_MONOTONIC, which had been macOS's number under a comment saying so. Then the renderer, on SDL_RenderGeometry rather than SDL_RenderFillRect: a batch opened, quads carrying their own colour, flushed — so a ROTATED quad is one call, which the old rectangle fill could not draw at any angle. Simp's own shape was verified from primary sources and SDL_Vertex's 20 bytes measured with a cc-compiled offsetof before a line of the module existed.
Per-OS support	done	DONE in two ADRs (ADR-0183, ADR-0184) — and this row read 'partial / THIN' one wave ago, which is why it is worth reading. It named the enabling gap as ONE parser arm and it was right: #insert was absent from the file-scope directive dispatcher, so comptime code could generate statements but not DECLARATIONS. What it got WRONG is what it inherited from the plan — that the remaining blocker was the circular per-OS library NAME. Two shell commands settled it: cc -lOpenGL fails on macOS, cc -framework OpenGL succeeds, and jr-link could emit only -L and -l. So a perfect name mechanism would have produced a name that does not link, and the real first blocker was a missing ARGUMENT FORM. Both are now built: #framework with LinkKind interned into the pool's library value (so the two forms are different values, no inference and no fallback), and #insert at file scope with generated items allocated straight into the file's arena. modules/GL picks its library AND its link form per OS in ordinary Jairs — three names, two argument forms — and the integration test reads otool -L rather than trusting an exit code. File's hedged O_* flags are unhedged. Socket's constants and Thread's pthread sizes are next and need no compiler work. A computed operand may generate only a library declaration (E0294): a phase order, not a policy, since it expands after const-eval and so is too late for a generated constant's value or a procedure's signature.

What the last stretch did, and the three claims it settled

FIVE LANGUAGE UTILITIES THE PLAN HAD OWED

ADR-0190 to 0194. Tests **hold at 1082**, corpus 270 to **279**, ADRs 189 to **194**, one new diagnostic code (E0295). Typed constants `FLAG : u32 : 256` — twenty casts gone from `modules/GL`; a pointer type as an intrinsic's argument; `type_of(x)`; an enum's **member names** and a view's elements in reflection; and **array literals** `s64.[1, 2, 3]`, which real Jai code uses 39 times and which was the most used construct this language lacked. Each wave paid the next: two arms added to `described_type` are why the array literal's element type cost no code, and `Point.[…]`, `(*u8).[…]` and `type_of(x).[…]` all work for free.

The signatures came from source, not documentation, and that is why the previous attempt was wrong. Jai's module tree is unpublished, but two open-source projects vendor it verbatim, so both copies were read and diffed against each other. Eight signatures were wrong. Two not cosmetically: the coordinate origin was mirrored, and every call carried a state handle Jai does not have. Removing it needed file-scope mutable variables in all three engines first.

The exhaustive-match rule has a hole, and it is a `let-else`. Adding `PlaceBase::Global` made nine sites in `jr-mir` fail to compile, each having to decide what a global means. The tenth was a `let ... else`, so it compiled silently and skipped globals by luck — and the wrong answer there would have been a real miscompile, because forwarding a store to a global across a call drops the store the callee was meant to see.

AN IMPORTED TYPE'S NAME, AND NAVIGATION FROM A TYPE POSITION

ADR-0200, found by a screenshot. An inlay hint read `window: structDeclId(1:1)` — the **eleventh** internal identifier here to reach a place a person reads. `FileSignatures` keys type names per **file**, so an importing file had no entry for an imported struct. The name lives in the **pool** now, beside `soa_counts`, whose own comment already made the argument word for word: the pool "is the one place every file's declarations already meet".

Two consumers wanted the same missing fact and had each worked around it. ADR-0171 recorded the DWARF struct DIE as anonymous, its §"honest gaps" naming this exact absence — so the editor showed an internal identifier and the debugger showed nothing, for one reason. Closing both with one map is the argument for putting it there. And the last resort matters as much as the lookup: it reads `<struct>` now, because a wrong answer presented as an answer is worse than an absent one.

A second, unrelated defect came from the same report: hover and goto-definition on a type annotation answered nothing at every column, because a `TypeRef` carries no span (ADR-0013) and no type resolution reaches `ResolveMap`. Read from the CST instead — giving `TypeRef::Name` a span was 19 sites across nine crates for what the tree already holds. Sema's own bookkeeping map looked like the answer and was **measured empty** for a

THREE CLAIMS THE PROBES SETTLED

A negative result from one program is evidence about that program. ADR-0172 §3 concluded, from a single test, that a structure-typed local could never be shown by name. ADR-0174 disproved it an hour later with a one-line change: it depends on **use**. One passed to a procedure by value is named, in both engines; one only assigned field by field is not. Generalising a negative needs a second program that differs in the suspected dimension.

An impossibility claim is a probe you have not run. ADR-0164 recorded that an event loop was impossible because `SDL_Event` is a union and no pointer to one could be taken. ADR-0165 built it the next wave — the premise was simply wrong, and it had been **written down** as a design constraint. Both the claim and its withdrawal are in the record, because a project that quietly deletes its wrong claims cannot learn the shape of them.

A data race is worth measuring, not asserting. The memory model's data-race clause is not a promise: the same three-thread program with a plain `+ 1` instead of an atomic add produced **1000 instead of 3000** on one run of three. Two thousand increments lost, no diagnostic. That number is why §3 of ADR-0177 exists as a section rather than a paragraph saying one would be written later.

body-local annotation: it would have worked for a parameter and silently failed for a local.

COMPLETION OF NAMES YOU HAVE NOT IMPORTED — AND A SECOND EDITOR

ADR-0199. Typing `create_window` with no `\#import "Window";` offered **nothing**. It is offered now, inserting a call snippet with the declaration's real parameter names, showing the whole signature, and adding the import in the same undo step. The server **formats** over the protocol too, so no editor needs a formatter command — and `editors/zed/` is a second supported editor.

The blocker was a missing line in the CLI, not anything in completion. `jr lsp` was the one subcommand of six that never pushed its bundled `modules/`, so with no explicit `--module-path` the server's search paths were **empty** — and `module_file` probes only those. It could resolve no `\#import` at all, which means the auto-import quick fix had been **silently dead** in a default invocation since it shipped. When a feature offers nothing, check its inputs are non-empty before reading its logic: an absent offer is indistinguishable from "there is nothing here".

Two defects in the existing path came out of writing the new one, both the same shape — completion read another file's raw HIR items where the code-action path read `file_exports`. So `\#scope_module` names were offered and sema then rejected what the editor had suggested, and an aliased import contributed bare names that only resolve qualified. And **a claim expired in the direction that reverses a decision**: ADR-0025 §3 refused to commit the generated `parser.c` because the gate regenerates it, but Zed compiles that file directly and never runs `tree-sitter generate`. Tracking it **strengthens** gate 6, whose drift check had always been blind to an ignored file.

BUILD SCRIPTS WRITTEN IN JAIRS — AND MEASURED AGAINST 23 REAL ONES

ADR-0195 to **0197**. Tests 1082 to **1103**, corpus 279 to **280**, ADRs 194 to **197**. `jr build build.jr` compiles the script, runs it in the VM and performs the compilations it asked for — no flag, because importing `modules/Compiler` is what makes a file a build script. Both of Jai's spellings work: a `main` script the driver runs, and a bare `#run` at file scope.

ADR-0195 §2 was wrong and ADR-0196 says so. It claimed a `#run` "cannot read a file, shell out, print, or even allocate"; two of the five were false, because `ffi.rs` has served `malloc` from the VM's own region since ADR-0061 and the refusal was keyed on the `#foreign` **declaration** rather than on whether a host is reached. Jai agrees, read from vendored source: its comptime `context.allocator` is an ordinary module, and **no allocator** is among the four `\#compiler` declarations that exist.

ADR-0197 then asked whether this is capable of the same work as a real `build.jai` — and measured rather than answered. It was not. `Build_Options.output_type` is set by **13 of 23** scripts and this compiler could not build a library at all. Both kinds link now, verified by linking C against an archive and a dylib. Two findings generalise: a `bool` was the wrong shape for the entry point (a static archive with a `main` fails a C link), and `\#program_export` was wired correctly everywhere while the symbol stayed **absent from the archive**, because reachability walks from `main` and a library has none — an export must be a **reachability root**.

Re-running that inventory against its own result found four more gaps, and one of them is a lesson. ADR-0198. The corpus program handed `"PATH".data` to `getenv` and got null — in **both** engines, because a string literal has no NUL — so the differential harness could never have caught it, and only running the program did. `to_c_string` exists because of that; with `to_string` and `c_style_strlen` it is the whole FFI boundary, one direction borrowing and the other unable to.

Writing the float scanner found a latent SSA defect — the fourth surfaced by a library rather than by a compiler test. `try_remove_trivial_phi` cascades, and returned a replacement its own cascade could remove, so a `goto` carried a parameter that no longer existed. The first fix was **wrong**: reserving placeholder arguments made an unfinished parameter look finished to the code whose job is asking whether it is finished. And two of ADR-0197′s five refusals were withdrawn — `BuildCpp` and a custom link command **compose**, verified by running the artefacts.

The flat namespace forced two renames, and both landed on Jai's own names. `Basic` gained a `u8` `to_upper` and `String` a `[]string` `join`, so two corpus programs got E0211 on every use; `to_upper_copy` and `path_join` are what Jai calls these exact procedures. And `modules/String` had no `trim` at all — found by a build script stripping a newline off `git rev-parse`, not by an audit. The message loop stays **refused**, not missing: Jai interleaves because its compiler is threads and a queue, this one is a memoised query engine.

PER-OS SUPPORT BECOMES LIBRARY CODE — AND THE SIMP PROGRAMME BEFORE IT

ADR-0183 and 0184. Test count 1073 to **1076** (1080 with LLVM in), corpus 262 to **266** files, ADRs 182 to **184**. `jr-link` learned `-framework` beside `-l`, and `#insert` learned file scope, so a module selects a library, a link form, a flag or a value per operating system in ordinary Jairs. `modules/GL` is the proof: three library names and **two argument forms**, chosen by a `#run` that reads `os()`.

Two shell commands demolished the plan's stated blocker. It ruled OpenGL out on a circular library **name**; `cc -lOpenGL` fails on macOS and `cc -framework OpenGL` succeeds, and the linker could emit only `-L` and `-l` — so a perfect name mechanism would have produced a name that does not link. And **a comment had expired**: a phase skipped recomputing signatures "because #insert adds no items", true only while an insert could not add declarations, so a generated procedure had none and the failure blamed its caller.

THE SIMP PROGRAMME, ON TOP OF TWELVE CLOSED WAVES

ADR-0179 through 0182. Test count 1071 to **1073**, corpus 255 to **262** files. Qualified imports, the target OS as a compile-time value, a per-OS clock id, and the graphics modules restructured onto `SDL_RenderGeometry` — which ADR-0187 has since replaced with OpenGL.

Its plan was wrong in six places, and every one was found by writing the thing. Two made items unbuildable as written. The plan's design for a qualified value would have taught nineteen sites a new shape; carried on the **name** instead, nothing downstream changed. It reserved a diagnostic code for a condition that turns out unreachable, so the code was **refused** rather than shipped as a promise nothing checks. It called for a salsa input for a value that cannot change in-process, costing a parameter at 50 call sites. It named the wrong cause for the file-scope gap — the fold was computed and thrown away one phase earlier, so its proposed fix changed nothing. It assumed module-level mutable state, **which this language does not have**. And it ruled OpenGL out on a library-name cycle when the real first blocker is that `jr-link` cannot emit `-framework` at all.

The wave order was right and the wave contents were not. W10 was listed as pure library work; it needed three compiler waves first (FFI aggregates, C-variadics, a library search path). W9′s "richer DWARF" was one line and is two implementations. W11′s blocker was a missing **type**, not the runtime work the row named. In each case the correction came from writing the thing, and in each case the plan row now records what was wrong rather than quietly reading as though it had always said this.

A seam drawn early keeps paying. ADR-0009 confined `cranelift-*` behind `jr-codegen::Backend` twelve waves before a second back end existed. The DWARF waves are the third collection to benefit: `TrapKind`, `SourceInfo::position` and now the whole span vocabulary live in the shared crate, so a line-table row and a trap's `--> file:line:col` come from **one** resolution and cannot say 41 and 40.

SETTLED SINCE THE LAST DASHBOARD

All twelve waves are closed, and everything is merged. Twenty-three more waves went onto `main` with `--no-ff`, one merge commit each, so `git log --merges main` still reads as the wave history — a claim you can check rather than a revision that goes stale the moment this file is committed. (That is why no SHA appears here: the commit that would record one changes the thing it records.) The branch names were corrected first **again**: sixteen waves had accumulated on a single `feat/c-variadic` branch, the same hygiene slip the last dashboard recorded fixing for two W8 sub-waves, so each now names its own work.

The one open slice criterion is a Linux x86-64 CI run someone has read. `main` was pushed for the first time on 2026-09-03, so it is no longer blocked on the push — but triggering a run is not reading one, and the outcome has not been observed even once. The Linux leg of the test matrix is the only thing that has ever executed this compiler on x86-64, so a genuine endianness or layout assumption surfaces there and nowhere else. Owed and specified rather than vague: a per-thread shadow call stack, so a trap in a spawned thread names the right frames; a register-resident local in DWARF, measured reachable and needing `.debug_loclists`; the security audit's remaining two dispatches (by hand — six subagent dispatches returned empty); and `tree-sitter test` added to gate 6, which catches grammar drift but not a broken rule.

Sources: PLAN.md §1.5 and §7, the ADR directory, `docs/decisions/DECISIONS.md`, and all seven gates run today on `main` **after** the merge, not on a branch. Every number was measured rather than carried forward — the test count from a full workspace run (1082, zero failures) and a second under `--features jr-cli/llvm` (1088), the corpus count from a file walk (279 `.jr` files under `tests/corpus/` outside `tests/corpus/modules/`), the ADR count from `docs/adr/`, and the editor-check count from a real `verify.lua` run (170). The diagnostic count is **every** `const NAME: &str = "E0nnn"` across the crates, which is 131 — and the counting rule is written down here because the previous number was 126 by a method nobody recorded: three of the 130 are named for what they mean rather than for their code (`jr-mir`'s `USE_OF_UNINITIALISED`, `MISSING_RETURN`, `JUMP_OUTSIDE_LOOP`), so a count that keys on the **name** misses them. Ownership is enforced by `jr-cli/tests/codes.rs` rather than asserted in prose. Rebuild with `typst compile jairs-dashboard.typ`.